

## Kapak

**Ürün Adı:** TelemetryAI

**Kapsam:** Unreal Engine Telemetry Plugin + (opsiyonel Unity) + Java Backend + React Web Panel

**Amaç:** Oyun içi telemetry verilerini toplayıp anlamlandırmak ve bu verilerden AI ile **kanıta dayalı** rapor üretmek (tavsiye vermeden, yorumlama/özetleme).

**Sürüm:** v1.0

**Tarih:** 2026-01-20

**Dil:** Türkçe (ASCII uyumlu içerik; kod örnekleri İngilizce)

## PROJE AÇIKLAMASI

TelemetryAI; Unreal Engine (ilk etap) ve opsiyonel olarak Unity oyun motorları için geliştirilecek, oyun içi telemetry verilerini performansı etkilemeyecek şekilde toplayıp Java tabanlı bir backend'e batch olarak gönderen, React tabanlı web panel üzerinden bu verileri analiz edip raporlayan bir üründür. Firma/organizasyon yapısı (org → project → environment: dev/stage/prod) ile çok kiracılı çalışır; her proje tek bir oyunu temsil eder ve erişim API key üzerinden sağlanır. Sistem "Core + Custom Events" yaklaşımını kullanır: temel zorunlu alanları olan event'ler (eventId, timestamp, sessionId, playerId, buildVersion, platform vb.) yanında, kullanıcı stüdyonun kendi tanımladığı şemaya göre eventName, eventDescription ve properties alanlarıyla esnek event'ler desteklenir. Şema yönetimi Unreal Editor UI ile değil; JSON import/export ve backend üzerinde versiyonlu schema kaydı + plugin'e sync ile yapılır.

Backend, gelen event'leri schema kurallarına göre doğrular; geçerli event'leri PostgreSQL'de saklar, geçersiz veya riskli (schema dışı, eksik alan, type uyumsuzluğu, payload limit aşımı, PII tespiti vb.) event'leri ayrı bir karantina tablosuna alır ve panelde ayrı ekranda gösterir. Düzenli aralıklarla (günlük/haftalık) agregasyonlar üretilir ve OpenAI API ile sadece bu düzenli/agregat veriler üzerinden "explanation" yönergesine uygun şekilde yorumlayıcı raporlar oluşturulur; raporlarda her bulgu, dayandığı metriklerle kanıtlanır. Plugin tarafında ana öncelik oyun performansıdır: game thread sadece event'i kuyruklar; JSON hazırlama, gzip, HTTP gönderim, retry/backoff ve offline spool (JSONL) background işleyicilerle yürütülür. Bu proje tek satış modeliyle dağıtılır; plan/kota ayrımı yoktur, ancak abuse ve stabilite için teknik limitler (batch boyutu, rate limit, payload limit) uygulanır.

## İçindekiler

1. Ürün Özeti ve Hedefler
  2. Kapsam (MVP) ve Kapsam Dışı
  3. Kararlar ve Kesinleşen Varsayımlar
  4. Genel Mimari
  5. Tenant (Org/Project/Environment) Modeli
  6. Telemetry Veri Modeli (Core + Custom)
  7. Custom Schema (JSON) Yönetimi (Import/Export + Sync)
  8. Ingestion API Sözleşmesi
  9. PostgreSQL Tasarımı (Partition/Retention)
  10. Invalid Event Karantina Akışı
  11. Analitik & Agregasyon Tasarımı
  12. AI Raporlama (OpenAI) Tasarımı
  13. Frontend (React) Modülleri
  14. Unreal Plugin Tasarımı (Performans Odaklı)
  15. Unity Plugin (Opsiyonel)
  16. Güvenlik, KVKK/GDPR, PII Politikası
  17. Dağıtım, Operasyon, Gözlemlenebilirlik
  18. Test ve Kalite Stratejisi
  19. Geliştirme Adımları (Sıralı Uygulama Planı)
- Ekler: Payload örnekleri, isimlendirme kuralları, hata kodları

## 1) Ürün Özeti ve Hedefler

### 1.1 Problem

Oyun stüdyoları oyun içi veriyi topluyor ama:

- veriler düzensiz (her event farklı format),
- performans kaygısı var (runtime yavaşlatma),
- raporlama/yorumlama zor,
- AI kullanımı kontrolsüz olursa “kanıtsız yorum” riski doğuyor.

## 1.2 Çözüm

TelemetryAI:

- Unreal (ve opsiyonel Unity) plugin ile event toplar,
- backend'e batch olarak gönderir,
- PostgreSQL'de düzenli saklar,
- agregasyon + AI rapor pipeline ile **kanıta dayalı** rapor üretir,
- web panelde geliştirici ekibe gösterir.

## 1.3 Ürün İlkeleri (Non-functional)

- **Performans:** oyun sırasında FPS/latency etkisi minimum; game thread bloklanmaz.
- **Güvenilirlik:** offline queue + retry/backoff ile event kaybı minimize edilir.
- **Şeffaflık:** AI raporu “kanıt metrikleri” ile birlikte sunulur.
- **Esneklik:** Core event seti + custom schema ile her oyun tipine uyarlanır.
- **Tek satış modeli:** Free/Pro/Ultra yok; tüm kullanıcılar aynı haklara sahip (ancak abuse önlemek için teknik limitler uygulanır: rate limit/payload limit vb. “plan” değildir).

## 2) Kapsam (MVP) ve Kapsam Dışı

### 2.1 MVP Kapsamı

**Backend (Java/Spring Boot):**

- Org/Project/Environment yönetimi
- API Key yönetimi (create/rotate/revoke)
- Schema yönetimi (JSON import/export, backend'de düzenleme)
- Event ingestion (batch), doğrulama, karantina (invalid)
- Temel dashboard agregasyonları (event sayıları, funnel temel)
- AI rapor üretimi (günlük + haftalık job)

**Frontend (React):**

- Login + Org/Project ekranları

- API key ekranı
- Schema yönetim ekranı (import/export, versiyon)
- Dashboard (basic)
- AI rapor ekranı (günlük/haftalık)
- Invalid/Karantina ekranı

#### Unreal Plugin:

- Config (endpoint, apiKey, env, buildVersion, flush ayarları)
- Core + Custom event gönderimi
- Batch queue, retry/backoff, offline queue
- Hem Blueprint node hem C++ API

#### Unity Plugin (opsiyonel):

- Unreal stabil olduktan sonra; Unity 2021 LTS hedefi

### 2.2 Kapsam Dışı (MVP’de yok)

- Gerçek zamanlı stream analytics (Kafka vb.)
- A/B test platformu
- Session replay video / heatmap gibi ağır özellikler
- Marketplace otomatik receipt verification (ilk etapta opsiyonel/sonrası)
- Multi-region mimari (ileride)

## 3) Kararlar ve Kesinleşen Varsayımlar

### 3.1 Kesin kararlar (senin verdiğin)

1. **Plan yok:** Free/Pro/Ultra yok. Tek satış; herkes aynı kullanım hakları.
2. **Schema oluşturma Unreal Editor UI ile değil:** JSON + import/export + backend’de düzenleme + plugin’e sync.
3. **Invalid event:** karantinaya alınır (ayrı tablo) ve panelde ayrı yerde gösterilir.

### 3.2 Teknik varsayımlar (şimdilik sabitliyoruz)

- Backend: Java 21 + Spring Boot 3.x

- DB: PostgreSQL (JSONB + partition)
- Frontend: React + TypeScript
- AI: OpenAI API (rapor bazlı, event bazlı değil)
- Dağıtım: Backend Railway (veya benzeri), Frontend Vercel; DB yönetilen Postgres (Railway/Supabase/Neon vb.)

### 3.3 “Tek satış + SaaS” risk notu (teknik)

Plan yok ama şu **teknik güvenlik limitleri** şart:

- max batch size
  - request rate limit (apiKey bazlı)
  - properties JSON boyut limiti
  - rapor job’larında token limiti / timeout
- Bu limitler “plan” değildir; altyapıyı ayakta tutmak içindir.

## 4) Genel Mimari

### 4.1 Bileşenler

- 1. Unreal Plugin**
  - a. Oyun içinden event üretir ve kuyruğa alır
  - b. Batch flush ile HTTPS üzerinden backend’e gönderir
- 2. Backend API (Ingestion + Yönetim + Rapor)**
  - a. API key doğrulama
  - b. Schema doğrulama
  - c. Valid event -> events tablosu
  - d. Invalid event -> quarantine tablosu
- 3. Aggregator/Jobs**
  - a. Günlük/haftalık agregasyonlar
  - b. AI rapor job’ları
- 4. Frontend Panel**
  - a. Proje yönetimi, schema yönetimi
  - b. Dashboard + AI raporlar + karantina ekranı
- 5. OpenAI Entegrasyonu**
  - a. Aggregated veriyi + explanation yönergesini gönderir

b. JSON (internal) veya metin (UI) çıktısı üretir

## 4.2 Veri akışı (özet)

Oyun -> Plugin Queue -> Batch POST /events/batch ->

Backend Validate (apiKey + schema) ->

Valid: events -> agregasyon -> AI report -> UI

Invalid: quarantine -> UI

# 5) Tenant (Org/Project/Environment) Modeli

## 5.1 Tanımlar

- **Org (Organization):** Firma/ekip
- **Project:** Tek bir oyun projesi (karar: project başına 1 oyun)
- **Environment:** dev / stage / prod (zorunlu)
- **API Key:** environment bazlı üretilir (öneri: prod ve dev ayrı key)

## 5.2 Erişim modeli (RBAC)

Minimum roller:

- **Admin:** her şeye erişir (key rotate, schema edit, report view)
- **Viewer:** sadece dashboard/report/karantina görüntüler

## 5.3 Kimlikler

- orgId, projectId backend tarafından üretilen UUID (veya ULID)
- environment: enum string: dev | stage | prod

## 6) Telemetry Veri Modeli (Core + Custom)

### 6.1 Core Event alanları (zorunlu)

Aşağıdaki alanlar tüm event'lerde olmalı:

- eventId (UUID): idempotency için zorunlu
- orgId, projectId (server tarafında apiKey'den resolve edilir; client göndermese de olur)
- environment (apiKey'den resolve edilir; client göndermese de olur)
- timestampClient (ms epoch)
- timestampServer (server set eder)
- eventName (string)
- eventDescription (string) — **schema'daki açıklama**; event gönderirken şart değil (server schema'dan doldurabilir)
- buildVersion (string)
- platform (string: Windows/Mac/Linux/Console/Mobile vb.)
- device (json: model, gpu, cpu, ram gibi; opsiyonel ama önerilir)
- playerId (string): anonim/hashed veya plugin-generated
- sessionId (string/uuid)
- properties (json) — custom numeric/string values

Not: org/project/env bilgisi güvenlik için **API key'den türetilmeli**. Client'ın gönderdiğini “doğru kabul etmek” risklidir.

### 6.2 buildVersion açıklaması

buildVersion: oyun build'ini tanımlayan sürüm string'i.

Amaç: “hangi build'den sonra drop arttı?” analizleri.

Örnek formatlar:

- 1.2.0
- 1.2.0+451
- 1.2.0+451.g1a2b3c4 (semver + build no + commit)

### 6.3 playerId hangi veriye denk gelir?

Güvenli seçenekler:

- **Plugin-generated Anonymous ID:** İlk çalışmada UUID üret, cihazda sakla. (PII içermez)
- **Platform ID hashed:** SteamID/EOS ID gibi stable bir id varsa SHA-256 hashleyip gönder. (ham hali gönderilmez)

Kural: playerId **PII olmamalı**. Email/telefon/ad gibi alanlar yasak.

## 6.4 Core event seti (MVP önerisi)

Bu set “zorunlu” değil; önerilen minimum:

- session\_start
- session\_end
- level\_start
- level\_end (properties: success/fail, duration, deaths, retries)
- error / exception (properties: code, messageHash, stackHash)
- crash (varsa)

“performance\_sample” default olarak **kapalı**. Eğer ileride istenirse **sampling + throttle** ile opsiyonel yapılır.

## 6.5 Custom events

Geliştirici istediği event’i basabilir:

- eventName: custom\_\* veya serbest string
- properties: tamamen serbest ama schema ile uyumlu olmalı

# elemetryAI Teknik Doküman v1.0 — Parça 2/4

## 8) Ingestion API Sözleşmesi

### 8.1 Kimlik doğrulama

- Tüm ingestion isteklerinde header zorunlu:
  - X-API-Key: <env\_api\_key>



- Backend tarafında API key şu bilgileri resolve eder:
  - orgId, projectId, environment, keyStatus

**Kural:** Client payload içinde orgId/projectId/environment gönderse bile **güvenilmez**; server API key'den türetir.

## 8.2 Endpoint: Batch Event Ingestion

**POST** /v1/events/batch

### *Request headers*

- X-API-Key: zorunlu
- Content-Encoding: gzip (opsiyonel ama önerilir)
- Content-Type: application/json

### *Request body (örnek)*

```
{
  "client": {
    "sdk": "telemetryai-unreal",
    "sdkVersion": "0.1.0",
    "engine": "unreal",
    "engineVersion": "5.3",
    "buildVersion": "1.2.0+451.g1a2b3c4",
    "platform": "Windows"
  },
  "events": [
    {
      "eventId": "b6e5d1b6-2df3-45dd-bd19-0c26f8b7d5a0",
      "timestampClient": 1737372000123,
      "eventName": "level_end",
      "sessionId": "0b2b3f2d-2e60-4b11-9a30-0c1f5f0b2f77",
      "playerId": "a9d2...hashed",
      "properties": {
        "levelId": "2",
        "result": "quit",
        "durationSec": 93.2,
        "deathCount": 4
      }
    }
  ]
}
```

```

    }
  }
]
}

```

### **Server davranışı**

1. API key doğrula, org/project/env resolve et
2. JSON parse + temel validasyon
3. Schema doğrulama:
  - a. eventName schema’da yoksa -> invalid/quarantine
  - b. property required eksikse -> invalid/quarantine
  - c. type uyumsuzsa -> invalid/quarantine
4. Idempotency:
  - a. eventId daha önce işlendi ise “duplicate” sayılır, tekrar yazılmaz.
5. Valid event -> events tablosuna insert
6. Invalid event -> quarantine\_events tablosuna insert (nedenleriyle)
7. Response: accepted/invalid/duplicate özet

### **Response body (örnek)**

```

{
  "received": 120,
  "accepted": 110,
  "invalid": 8,
  "duplicate": 2,
  "invalidSamples": [
    {
      "eventId": "....",
      "eventName": "custom_unknown",
      "reasons": ["EVENT_NOT_IN_SCHEMA"]
    }
  ]
}

```

### **HTTP status kodları**

- 200 OK: batch işlendi (accepted/invalid olabilir)
- 400 Bad Request: body parse edilemedi / aşırı büyük / zorunlu alanlar yok
- 401 Unauthorized: api key yok/yanlış
- 403 Forbidden: key revoked / org disabled
- 413 Payload Too Large: limit aşıldı
- 429 Too Many Requests: rate limit
- 500: server hatası (plugin retry uygular)

### **8.3 Limitler (plan yok ama sistem güvenliği için)**

MVP öneri değerleri (config):

- Max request body: 1–2 MB (gzip ile daha düşük olur)
- Max event count per batch: 200–500
- Max single event properties JSON size: 16–32 KB
- Rate limit: key başına saniyede N request (örn. 2–5 rps)
- Retry/backoff: plugin tarafında exponential

### **8.4 Event “kanonlaştırma” (normalizasyon)**

Server insert öncesi şunları set eder:

- `timestampServer = now()`
- `orgId/projectId/environment = fromApiKey`
- `eventDescription = fromSchema` (ister event'te gelsin ister gelmesin)

## **9) PostgreSQL Tasarımı (Partition/Retention)**

### **9.1 Neden PostgreSQL (JSONB ile)**

- Tenant/kimlik/erişim gibi ilişkisel ihtiyaçlar güçlü
- JSONB ile event properties esnek
- Partition + indeks ile yüksek hacim yönetilebilir
- Gerekğinde TimescaleDB/ClickHouse'a evrilebilir

## 9.2 Temel tablolar

### 9.2.1 Organizasyon ve erişim

- orgs (id, name, created\_at, status)
- users (id, email, password\_hash, created\_at)
- org\_members (org\_id, user\_id, role)

### 9.2.2 Proje ve anahtar

- projects (id, org\_id, name, created\_at)
- environments (id, project\_id, env\_name) -> dev|stage|prod
- api\_keys (id, project\_id, env\_name, key\_hash, created\_at, revoked\_at)

key\_hash: düz api key DB'de tutulmaz. Hash + salt saklanır.

### 9.2.3 Schema

- schemas (id, project\_id, version, json, created\_at)
- schema\_events (schema\_id, event\_name, event\_description, required) (opsiyonel normalizasyon)
- schema\_event\_properties (schema\_id, event\_name, key, type, required, allowed\_json) (opsiyonel)

MVP'de iki seçenek:

- A) Şemayı schemas.json içinde full tut (en basit)
- B) Şemayı normalleştir + JSON (daha hızlı validation + query)

Bu dokümanda MVP için: **A + minimal yardımcı index.**

### 9.2.4 Events (valid)

- events (partitioned by day/week)
  - id (bigserial)
  - org\_id
  - project\_id
  - env\_name
  - event\_id (uuid) UNIQUE (org+project+env scope da olabilir)

- event\_name
- event\_description
- ts\_client (timestampz)
- ts\_server (timestampz)
- player\_id (text)
- session\_id (text)
- build\_version (text)
- platform (text)
- device (jsonb)
- properties (jsonb)

#### Önerilen indeksler:

- (project\_id, env\_name, ts\_server)
- (project\_id, env\_name, event\_name, ts\_server)
- (project\_id, env\_name, build\_version, ts\_server)
- (project\_id, env\_name, player\_id, ts\_server) (dikkat: yüksek cardinality)

#### Partition stratejisi:

- Günlük partition (yüksek hacim için)
- Alternatif: haftalık partition (daha az partition)  
MVP: **haftalık** daha pratik; hacim büyürse günlük.

#### 9.2.5 Quarantine (invalid)

- quarantine\_events
  - id (bigserial)
  - org\_id, project\_id, env\_name
  - received\_at (ts)
  - event\_id (nullable)
  - event\_name
  - raw\_event (jsonb) // gelen ham event
  - reasons (jsonb) // ["EVENT\_NOT\_IN\_SCHEMA", "MISSING\_REQUIRED\_PROPERTY:levelId"]
  - schema\_version\_at\_time (int nullable)
  - client\_sdk, client\_sdk\_version, engine, engine\_version, build\_version, platform

Index:

- (project\_id, env\_name, received\_at)
- (project\_id, env\_name, event\_name, received\_at)

### 9.3 Retention (365 gün hedef)

- events tablosu: 365 gün
- quarantine\_events: 30–90 gün (opsiyon) (çünkü ham payload saklar, şişer)

Retention job:

- Partition drop ile hızlı silme (events)
- Quarantine için delete by date + vacuum

## 10) Invalid Event Karantina Akışı

### 10.1 “Invalid” tanımı

Aşağıdakilerden biri olursa:

- eventName schema’da yok
- required property eksik
- property type mismatch
- JSON boyut limitleri aşıldı
- schemaVersion uyumsuz (opsiyonel)

### 10.2 Karantina davranışı

- Invalid event **asla events tablosuna girmez**
- quarantine\_events tablosuna yazılır
- Nedenleri reasons[ ] şeklinde kaydedilir
- UI’da ayrı ekran: “Quarantine / Invalid Events”
  - filtre: eventName, reason, buildVersion, tarih
  - top reasons: bar chart (opsiyon)

## 10.3 Karantinadan kurtarma (MVP’de manuel)

MVP’de otomatik “replay” yok.

Ama kullanıcı schema’yı düzelttikten sonra:

- Karantinadaki eventler UI’da görülebilir
- İleride “Reprocess selected” özelliği eklenebilir (MVP dışı)

## 10.4 Validation Karar Matrisi (Kesin)

**Amaç:** Event’lerin *valid* mi *invalid/quarantine* mi olacağına tutarlı şekilde karar vermek.

### 10.4.1 Karar kuralları

Aşağıdaki kurallar **event bazında** uygulanır (batch içindeki her event ayrı değerlendirilir):

#### 1. Schema uyumsuzlukları → QUARANTINE

- Koşullar:
  - eventName schema’da yok
  - required property eksik
  - property type mismatch
  - allowed list ihlali
- Aksiyon:
  - Event **events** tablosuna yazılmaz
  - quarantine\_events tablosuna yazılır
  - reasons[ ] doldurulur

#### 2. Payload/limit ihlali → QUARANTINE

- Koşullar:
  - Tek event properties JSON boyutu limit üstü
  - string alan uzunluk limit üstü
  - batch limit üstü (max event count / body size)
    - Not: batch çok büyükse request 413 dönebilir; ancak mümkünse event bazında işleyip sadece sorunlu olanları quarantine’ye atmak tercih edilir.
- Aksiyon:
  - Event quarantine’ye gider
  - Reason: PAYLOAD\_TOO\_LARGE veya FIELD\_TOO\_LONG:<field>

#### 3. PII (kişisel veri) tespiti → QUARANTINE

- Koşullar:
  - properties içinde yasak anahtarlar (email, phone vb.)
  - değerlerde email/telefon benzeri pattern yakalanması (regex)
- Aksiyon:
  - Event quarantine'ye gider
  - Reason: PII\_DETECTED (+ opsiyonel detay: hangi alan)

#### 4. Duplicate (idempotency) → DUPLICATE (YAZMA)

- Koşul:
  - Aynı eventId daha önce işlendiyse (project+env scope)
- Aksiyon:
  - Event DB'ye tekrar yazılmaz
  - Response'ta duplicate++

#### 5. Valid → ACCEPT

- Koşul:
  - Yukarıdaki hiçbir kurala takılmıyor
- Aksiyon:
  - Event events tablosuna insert edilir

### 10.4.2 Standard reason kodları

reasons[ ] içinde kullanılacak ana reason listesi:

- EVENT\_NOT\_IN\_SCHEMA
- MISSING\_REQUIRED\_PROPERTY:<key>
- TYPE\_MISMATCH:<key>
- ALLOWED\_VIOLATION:<key>
- PAYLOAD\_TOO\_LARGE
- FIELD\_TOO\_LONG:<field>
- PII\_DETECTED

### 10.4.3 Batch response kuralı

Backend batch response her zaman şu alanları döner:

- received, accepted, invalid, duplicate
- invalidSamples[ ] (en fazla N adet örnek)



## 11) Analitik & Agregasyon Tasarımı

### 11.1 Neden agregasyon

Raw events üzerinden UI sorguları pahalı olur.

Bu yüzden scheduled job'lar ile özet tablolar tutulur.

### 11.2 MVP agregasyonları

- Günlük event counts:
  - toplam event
  - eventName bazında dağılım
- Session summary (opsiyon):
  - session duration (start/end eventlerinden)
- Funnel (level bazlı):
  - level\_start sayısı
  - level\_end (success/fail/quit)
  - conversion/drop metrikleri

### 11.3 Özet tablo örnekleri

- daily\_project\_metrics
  - date, project\_id, env\_name
  - total\_events
  - unique\_players (approx) (MVP: distinct count - pahalı olabilir)
  - sessions\_started
- daily\_event\_name\_metrics
  - date, project\_id, env\_name, event\_name
  - count
- daily\_level\_funnel
  - date, project\_id, env\_name, level\_id
  - starts
  - ends\_success
  - ends\_fail
  - ends\_quit
  - avg\_duration\_sec (from properties.durationSec)

## 11.4 DAU / “DAU nedir?” cevabı

- **DAU (Daily Active Users):** bir günde en az 1 event gönderen benzersiz oyuncu sayısı.
- MAU da aylık versiyonu.

Senin için pratik açıklama:

- “Bugün oyuna girip event atan kaç farklı oyuncu var?” = DAU.

Not: DAU için playerId gerekir. Eğer playerId güvenilir üretilmezse DAU anlamsızlaşır.

## 11.5 “Oyuncu başına kaç event/oturum?” ne demekti?

Bu, sistemin yükünü tahmin etmek için:

- 1 oyuncu bir oturumda kaç event yollar? (örn. 20)
  - günde kaç oturum yapar? (örn. 2)
- Bu sayılar ingestion kapasitesini belirler.

# 12) AI Raporlama (OpenAI) Tasarımı

## 12.1 Amaç

AI “tavsiye” vermez; sadece:

- Verideki paterni anlatır,
- Drop artışı / anomali / trend gibi çıkarımları özetler,
- “kanıt” olarak metrikleri listeler.

## 12.2 Rapor üretim periyotları

- Günlük rapor
- Haftalık rapor
- Manuel tetikleme: şimdilik yok (kararın)

## 12.3 AI input stratejisi

AI'ye ham event basmak yerine:

- aggregated metrikler
- top anomaliler
- buildVersion kırılımları
- quarantine top reasons gönderilir.

## 12.4 Explanation alanı (senin istediğin kritik kısım)

Backend AI çağrısına şu alanı ekler:

- explanation: "Veriyi nasıl yorumlaması gerektiğini belirten yönerge"

Örnek explanation:

- "Level 2 -> Level 3 geçişinden sonra oyuncuların oyunu bırakma oranını analiz et. BuildVersion bazında kıyasla. Sadece sayısal metriklerle kanıtla."

## 12.5 Prompt çıktısı (internal JSON + UI metin)

Sen UI'da düz metin göstereceğini söyledin.

Yine de backend tarafında **stabil** olması için AI'den JSON istemek daha güvenli (UI'ya metin olarak render edilebilir).

Bu yüzden tasarım:

- AI'den JSON al (findings/evidence)
- UI'da findings'i metne çevirip göster

Örnek internal output:

```
{
  "summary": "Son 7 gunde Level2->Level3 sonrasi quit orani artti.",
  "findings": [
    {
      "title": "Level2->Level3 drop artisti",
      "description": "Level3 start / Level2 end oraninda dusus goruldu.",
    }
  ]
}
```

```
"evidence": [  
  {"metric": "conversion_L2_to_L3", "before": 0.62, "after":  
0.41, "range": "2026-01-13..2026-01-19"},  
  {"metric": "quit_after_L2", "before": 0.18, "after": 0.33}  
]  
},  
],  
"limitations": ["playerId anonim; cihaz bazli olabilir"]  
}
```

## 12.6 Kanıtlama (UI'da güven)

UI'da rapor ekranında:

- “Bulgu” + altında “Kanıt metrikleri” listesi gösterilir.
- İlgili dashboard filtresine link verilir:
  - Örn. “Level=2, event=level\_end, result=quit, tarih=son 7 gün”

## 12.7 Maliyet ve güvenlik kontrolleri

- Token limit (rapor başına)
- Timeout ve retry (AI çağrısı)
- Fallback: AI başarısızsa “Rapor üretilemedi” + metrikler yine gösterilebilir

# TelemetryAI Teknik Doküman v1.0 — Parça 3/4

## 13) Frontend (React) Modülleri

### 13.1 Teknoloji seçimi (detay)

- **React 18 + TypeScript**
- Build: **Vite**
- State: basit başlamak için React Query (tanstack) + local state
- Router: react-router

- UI Kit: MUI veya AntD (birini seçmek şart; doküman “MUI önerir” ama zorunlu değil)
- Auth token saklama: `httpOnly` cookie tercih edilir. Eğer cookie zor ise `localStorage` (riskli, XSS’e açık) -> MVP’de kabul edilebilir ama güvenlik notu düş.

## 13.2 Ekranlar (MVP)

### 13.2.1 Auth

- Login
- Org seçimi (eğer bir kullanıcı birden çok org’da ise)

### 13.2.2 Project Management

- Project list
- Project create
- Environment görüntüleme (dev/stage/prod sabit)

### 13.2.3 API Key Management

- Key list (dev/stage/prod)
- Key rotate (yeni üret, eskisini revoke et)
- Key copy (gösterim: sadece create anında full key göster, sonra maskeli)

### 13.2.4 Schema Management

- Current schema görüntüle (version dahil)
- JSON import (upload/paste)
- JSON export (kopyalanabilir text olarak)
- Schema edit (UI üzerinden değil): MVP’de “raw JSON edit” alanı olabilir
- Schema validation hataları (backend döndürür)
- “Sync info”: Plugin’in hangi `schemaVersion`’ı çektiği (opsiyon)

### 13.2.5 Dashboard

- Overview:
  - total events (range)
  - eventName dağılımı
  - funnel summary (level bazlı)

- Funnel view:
  - Level listesi (levelId)
  - starts / ends\_success / ends\_fail / ends\_quit
  - conversion & drop oranları
- Build comparison:
  - buildVersion kırılımında funnel farkları

### **13.2.6 AI Reports**

- Günlük rapor listesi
- Haftalık rapor listesi
- Rapor detail:
  - Summary metin
  - Findings listesi
  - Her finding altında Evidence metrikleri (tablo)
  - Evidence'den dashboard filtre linkleri

### **13.2.7 Quarantine / Invalid Events**

- Liste + filtre:
  - tarih aralığı
  - eventName
  - reason
  - buildVersion
- “Raw event detail” modal:
  - raw\_event JSON göster
  - reasons listesi

### **13.2.8 Ticket (opsiyonel)**

Sen “panelden ticket açılsın” dediğin için MVP’de çok basit:

- “Support ticket create” form (subject + message)
- Backend’e kayıt
- Admin user panelinden görüntüleme (superuser)

Not: Bu ürünün ana akışını büyütmesin. Basit tut.

## 14) Backend (Java) Tasarımı — Teknoloji ve Modül Detayı

### 14.1 Teknoloji seçimi (kesin)

- **Java 21**
- **Spring Boot 3.x**
- Build: Maven veya Gradle (tekini seç; Maven daha yaygın)
- DB: PostgreSQL
- Migration: Flyway
- JSON: Jackson
- Auth: Spring Security + JWT
- Scheduling: Spring Scheduler (MVP), ileride queue (RabbitMQ/Kafka)

### 14.2 Modüler yapı (monolith ama modül ayrımı)

MVP’de mikroservis şart değil; ama kodu modüllerle ayır:

- auth (login, jwt)
- tenant (org, users, membership)
- project (project/env/api keys)
- schema (import/export/current version)
- ingestion (events/batch)
- analytics (aggregations + dashboard endpoints)
- ai (report jobs + report endpoints)
- quarantine (invalid events endpoints)
- support (tickets - opsiyon)

### 14.3 Önemli endpoint seti (özet)

- Auth:
  - POST /v1/auth/login
  - POST /v1/auth/logout
  - GET /v1/auth/me
- Tenant/Project:
  - POST /v1/projects

- GET /v1/projects
- GET /v1/projects/{id}
- API Keys:
  - POST /v1/projects/{id}/keys/rotate?env=prod
  - GET /v1/projects/{id}/keys
- Schema:
  - GET /v1/projects/{id}/schema/current
  - POST /v1/projects/{id}/schema/import
  - GET /v1/projects/{id}/schema/export
- Ingestion:
  - POST /v1/events/batch
- Analytics:
  - GET /v1/projects/{id}/dashboard/overview?env=prod&range=7d
  - GET /v1/projects/{id}/dashboard/funnel?env=prod&range=7d
- AI Reports:
  - GET /v1/projects/{id}/reports/daily?range=30d
  - GET /v1/projects/{id}/reports/weekly?range=12w
- Quarantine:
  - GET /v1/projects/{id}/quarantine?env=prod&range=7d&eventName=...
  - GET /v1/projects/{id}/quarantine/{qid}
- Support (ops):
  - POST /v1/support/tickets
  - GET /v1/admin/tickets (superuser)

#### 14.4 Validation stratejisi (schema + payload)

1. **Temel zorunlu alanlar:** eventId, timestampClient, eventName, sessionId, playerId
2. **Schema check:**
  - a. eventName var mı?
  - b. required properties var mı?
  - c. type uyuyor mu?
3. **PII guard:**
  - a. properties içinde yasak anahtarlar: email, phone, name vs.
  - b. veya regex ile email/telefon pattern yakala (false positive olabilir; MVP'de sadece "warn+quarantine" tercih edilebilir)
4. **Boyut limitleri:**
  - a. properties json max size



b. string max length

## 14.5 Idempotency (eventId)

- eventId UNIQUE constraint (scoped by project+env)
- Duplicate gelirse:
  - accepted sayma, duplicate olarak response'a yaz

## 14.6 Scheduler Jobs (MVP)

- AggregationJob:
  - her gece 02:00 (server TZ) veya hourly
  - daily metrics tablolarını doldurur
- AIReportJobDaily:
  - her gün 03:00
- AIReportJobWeekly:
  - haftada 1 (Pazartesi 04:00)

Bu saatler config olmalı.

# 15) Unreal Plugin Tasarımı (Performans Odaklı)

Senin “en önemli şey FPS etkisi olmasın” şartına göre plugin mimarisi kesin:

## 15.1 Hedefler

- Game thread'de **minimum iş** (sadece enqueue)
- Ağ gönderimi async
- Batch + gzip
- Offline queue (disk)
- Retry/backoff
- “Drop-safe”: kuyruk dolarsa kontrollü davranış

## 15.2 Plugin API yüzeyi

Hem Blueprint hem C++:

**Blueprint nodes:**

- TelemetryAI\_Init(Config)
- TelemetryAI\_TrackEvent(EventName, PropertiesMap)
- TelemetryAI\_Flush()
- TelemetryAI\_SetPlayerId(PlayerIdOrNull) (opsiyon)
- TelemetryAI\_SetSessionId(SessionIdOrNull) (opsiyon, yoksa auto)
- TelemetryAI\_Shutdown()

**C++ API:**

- TelemetryAI::Init(const FTelemetryConfig&)
- TelemetryAI::TrackEvent(FName EventName, const TMap<FString, FTelemetryValue>& Props)
- TelemetryAI::Flush()

**15.3 Config (Project Settings)**

- endpoint URL
- apiKey
- environment (dev/stage/prod)
- buildVersion
- flush interval (sec)
- max queue size (event count)
- max batch size (event count)
- offline spool enable (true/false)
- spool path (default Saved/)
- gzip enable (true/false)

**15.4 İç mimari (thread/queue)****Bileşenler:**

- 1. EventQueue (in-memory)**
  - Lock-free veya minimal lock (MVP: mutex yeter)
  - Game thread TrackEvent çağırır -> queue push
- 2. Serializer (background)**
  - Queue'dan batch çeker

- JSON serialize (FJsonObject)
- Gzip (opsiyon)

### 3. Sender (HTTP async)

- Unreal HTTP module ile POST
- Response parse
- Başarısız -> retry/backoff

### 4. Offline Spooler

- Ağ yoksa veya 5xx ise:
  - batch'i disk'e yaz (\*.jsonl veya \*.bin)
- Sonra network gelince:
  - disk'ten sırayla okuyup gönder

## 15.5 Flush mantığı (batch ne zaman gider?)

Flush tetikleri:

- Timer: her flushInterval saniye
- Queue length >= maxBatchSize
- App background/exit (OnEndPlay)

## 15.6 “Runtime yavaşlatır mı?” sorusunun net cevabı

Bu mimaride:

- Game thread sadece queue.push(event) yapar
- JSON serialize + HTTP **ayrı thread** / async
- FPS etkisi pratikte çok düşer (event başına mikro maliyet)

**Yine de riskler:**

- Çok büyük properties JSON (o yüzden limit var)
- Çok sık event atmak (o yüzden throttle önerisi)

## 15.7 Event throttle / sampling (MVP politikası)

“Performance\_sample kötü örnek” dediğin için default yok.

Ancak geliştirciler bazen çok sık event basar.

Bu yüzden plugin'de opsiyonel:

- **ThrottleByEventName:** eventName başına saniyede max N
- **SampleRate:** yüzde (örn. %10 oyuncu)  
MVP’de kapalı gelir, ama altyapısı basit tutulur.

## 15.8 Schema ile uyum (plugin tarafında)

Plugin schema’yı (JSON) cache’ler.

Event atılırken iki seçenek:

- **Strict mode (önerilen):**
  - plugin event’i schema’ya göre kontrol eder
  - uymazsa local “invalid” log’a düşer ve göndermez
  - (Ama bu debug için iyi, veri kaybı yaratabilir)
- **Server authoritative (MVP önerisi):**
  - plugin sadece temel alanları kontrol eder
  - asıl doğrulama server’da yapılır
  - invalid olursa quarantine’de görünür

MVP’de: **Server authoritative** seçiyoruz (senin “panelde görelim” kararınla uyumlu).

## 15.9 Offline Spool (JSONL) — Kesin Kurallar

**Amaç:** Network yokken veya geçici hata varken event kaybetmeden diske yazmak ve sonra sırayla göndermek.

### 15.9.1 Format

- Offline spool dosya formatı: **JSONL (JSON Lines)**
- Her satır: **tek bir event JSON nesnesi**
- Dosya uzantısı: **.jsonl**

Örnek satır:

```
{"eventId":"...", "timestampClient":1737..., "eventName":"level_end", "sessionId":"...", "playerId":"...", "properties":{"levelId":"2", "result":"quit"}}
```

### 15.9.2 Dosya konumu

- Varsayılan dizin:
  - Unreal: Saved/TelemetryAI/spool/
- Alt dizinler:
  - spool/pending/ (gönderilmeyi bekleyen)
  - spool/sent/ (opsiyonel: başarıyla gönderilenler kısa süre tutulabilir)
  - spool/failed/ (opsiyonel: tekrar denenecekler)

### 15.9.3 Rotation (dosya döndürme)

Bir spool dosyası aşağıdaki koşullardan biri sağlanınca kapatılır ve yenisi açılır:

- Dosya boyutu > **5 MB VEYA**
- Satır sayısı > **10.000**

Dosya adı önerisi:

- spool\_YYYYMMDD\_HHMMSS\_0001.jsonl
- spool\_YYYYMMDD\_HHMMSS\_0002.jsonl

### 15.9.4 Yazma stratejisi (performans)

- Game thread **diske yazmaz**.
- Game thread sadece memory queue'ya event atar.
- Background worker, flush edemediği batch'i JSONL'e append eder.
- Append işlemi:
  - \n ile satır sonu
  - mümkünse buffered write

### 15.9.5 Gönderme stratejisi (FIFO)

- Plugin network tekrar sağlandığında:
  - pending/ klasöründeki dosyaları **FIFO** sırayla işler (dosya adına göre sıralı).
  - Her dosyayı satır satır okuyup batch'ler halinde /v1/events/batch endpoint'ine gönderir.
  - Bir batch başarıyla gönderilirse o satırlar "işlenmiş" kabul edilir.

**Dosya tam başarıyla biterse:**

- Dosya sent/ klasörüne taşınır **veya** silinir (MVP: silmek yeterli)

#### **Batch gönderimi başarısız olursa:**

- Retry/backoff devreye girer
- Dosya pending kalır, ileride tekrar denenir

#### **15.9.6 Retry/Backoff kuralı**

- 5xx / network fail: exponential backoff (örn. 1s, 2s, 4s, 8s... max 60s)
- 4xx (401/403): key invalid → retry yapma, local log + kullanıcıya uyarı
- 429: backoff uygula (server rate limit)

#### **15.9.7 Kuyruk dolarsa ne olur? (fail-safe)**

- Memory queue max dolarsa:
  - Varsayılan davranış: en eski event'i düşürme **YOK**
  - Öncelik: batch'i diske yazmayı hızlandır
- Eğer disk yazma da başarısız olursa:
  - Event drop edilebilir ama mutlaka local log yazılır (MVP: sayıcı artır, "dropped\_events\_count")

## **16) Unity Plugin (Opsiyonel)**

Unity 2021 LTS hedefi. Unreal stabil olduktan sonra.

MVP unity yaklaşımı:

- TelemetryAIConfig (ScriptableObject)
- TelemetryAI.Track(eventName, props)
- batch flush coroutine
- offline spool: persistentDataPath

- same ingestion endpoint

Eğer karmaşıklırsa ilk release sadece Unreal. Unity sonraya.

## 17) Güvenlik, KVKK/GDPR, PII Politikası (özet)

### 17.1 PII yasağı

Plugin/SDK tarafında ve backend’de:

- email/phone/name gibi PII alanlar gönderilmemeli
- regex ile tespit edilirse quarantine + uyarı

### 17.2 Data minimization

AI’ye ham event değil agregat veriler gönderilir.

### 17.3 API Key güvenliği

- DB’de sadece hash
- rotate mekanizması
- key leak durumunda revoke

### 17.4 GDPR/KVKK “hassasiyet” ne demek?

EU hedefliyorsan:

- kullanıcı verisi anonim olmalı
- saklama süresi (retention) tanımlı olmalı
- talep halinde silme (DSAR) gibi süreçler gerekebilir

MVP’de minimum:

- playerId anon/hased
- retention 365 gün (config)
- “delete by playerId” endpointi (opsiyon) — MVP dışı ama roadmap

## 17.5 PII Tespiti ve İşleme Politikası (Kesin)

**Amaç:** KVKK/GDPR hedefi nedeniyle telemetry içinde kişisel veri sızıntısını engellemek.

### 17.5.1 PII kabul edilmez

Aşağıdaki veri türleri gönderilmemelidir:

- email adresi
- telefon numarası
- gerçek isim/soyisim
- açık adres
- TCKN, pasaport vb.

### 17.5.2 Tespit yöntemleri

#### A) Anahtar bazlı (properties key)

Aşağıdaki anahtarlar (case-insensitive) tespit edilirse PII sayılır:

- email, e-mail
- phone, telephone, gsm
- name, surname, fullname
- address, location
- tc, tckn, identity, passport

#### B) Pattern bazlı (değer regex)

- Email pattern (örn. [something@domain.tld](mailto:something@domain.tld))
- Telefon pattern (ülke kodu + uzun sayı dizisi)

Not: Regex false-positive üretebilir. Bu yüzden “reject” yerine quarantine tercih edilir.

### 17.5.3 Aksiyon (MVP kararı)

- PII tespit edilen event:
  - **events** tablosuna yazılmaz
  - **quarantine\_events** tablosuna yazılır
  - `reasons[ ]` içine `PII_DETECTED` eklenir
- UI’da Quarantine ekranında “PII tespit edildi” etiketi gösterilir.



### 17.5.4 AI'ye veri gönderiminde kural

- AI'ye **ham event gönderilmez**, sadece agregat metrikler gönderilir.
- Quarantine içeriği AI'ye gönderilmez (gerekirse sadece sayısal özet: "PII\_DETECTED sayısı").

## 7) Custom Schema (JSON) Yönetimi (Import/Export + Sync)

### 7.1 Hedef

Kullanıcı firma:

- Kendi event şemasını tanımlar,
- Backend'e kaydeder,
- İsterse backend'de düzenler,
- Plugin tarafında şemayı güncelleyip (sync) doğru event göndermeyi garanti eder.

### 7.2 Schema konsepti

- Schema, proje + environment bazlı olabilir ama öneri: **proje bazlı** ve environment farkı olmadan tek şema (MVP).
- Schema versioning: schemaVersion artar (1,2,3...)

### 7.3 Schema JSON formatı (önerilen)

```
{  
  "schemaId": "uuid",  
  "schemaVersion": 3,  
  "projectId": "uuid",  
  "updatedAt": "2026-01-20T12:00:00Z",  
  "events": [  

```

```

{
  "eventName": "level_end",
  "eventDescription": "Level bitis olayi. Basari durumunu ve
sureyi gonder.",
  "required": true,
  "properties": [
    { "key": "levelId", "type": "string", "required": true },
    { "key": "result", "type": "string", "required": true,
"allowed": ["success","fail","quit"] },
    { "key": "durationSec", "type": "number", "required": true },
    { "key": "deathCount", "type": "number", "required": false }
  ]
}
]
}

```

## 7.4 Schema doğrulama kuralları

- eventName unique olmalı (case-insensitive çakışma yok)
- properties.key unique olmalı
- type sadece: string | number | boolean | object | array
- allowed sadece string/number tiplerde
- max property sayısı (MVP): örn. 50
- max description length: örn. 500 char

## 7.5 Import/Export akışı

- UI'dan JSON import -> backend validate -> kaydet -> schemaVersion++
- UI'dan export -> güncel şema JSON olarak indir (sen indiremediğin için bu yine dokümanda konsept olarak var; uygulamada export endpoint'i olur)

## 7.6 Plugin schema sync

Plugin çalışma anında:

- açılışta /schema/current çekebilir (cache)
- veya manuel "Refresh Schema" çağırısı
- schema değişirse plugin local cache'i günceller

**Cache/TTL:** 5-15 dakika (config ile)

## 18) Test ve Kalite Stratejisi (Detaylı)

Bu proje “plugin + backend + UI + AI” olduğu için testleri katman katman tasarlamazsak, hata nerede çıkıyor anlaşılmaz.

### 18.1 Test türleri ve hedefleri

#### 18.1.1 Backend Unit Tests

Hedef: iş kuralları doğru mu?

- Schema validation (eventName, required properties, type check)
- Quarantine reason üretimi
- Idempotency (eventId duplicate)
- API key doğrulama (hash compare)
- Aggregation hesapları (funnel, counts)
- AI input builder (agregat verinin doğru paketlenmesi)

Kapsam:

- JUnit 5
- MockMvc veya WebTestClient (Spring)
- Testcontainers (Postgres) önerilir ama MVP’de H2 ile başlayıp sonra Postgres’e geçebilirsiniz.

**Kabul kriteri:**

- Her kritik modül için minimum %70 coverage değil, **kritik fonksiyonların testli olması** (özellikle validation ve aggregation).

#### 18.1.2 Backend Integration Tests

Hedef: gerçek DB ve gerçek endpoint akışı çalışıyor mu?

- /v1/events/batch -> events insert
- invalid event -> quarantine insert
- schema import/export -> version artıyor mu?

- dashboard endpoint -> aggregate tablolarından dönüyor mu?

**Kabul kriteri:**

- “Happy path” + 3–5 hatalı senaryo (invalid, too large, unauthorized, rate limit).

**18.1.3 Contract Tests (SDK/Plugin ↔ Backend)**

Hedef: plugin payload değişince backend kırılmasın.

- JSON contract: `client + events[ ]` alanları
- Response contract: `accepted/invalid/duplicate`

Bu testleri basitçe:

- Backend repo içinde “sample payload JSON” dosyaları
- CI’da bu payload’larla endpoint testleri

**Kabul kriteri:**

- Payload formatı değişirse test kırılır ve fark edilir.

**18.1.4 Load / Performance Tests (Ingestion)**

Hedef: ingestion TPS altında patlıyor mu?

- k6 veya JMeter ile `/events/batch` yük testi
- hedef: ilk MVP için bile “saniyede X batch” ölçümü şart

Ölçümler:

- p95 latency
- error rate
- DB insert throughput
- CPU/RAM

**Kabul kriteri:**

- Belirlenen yükte 429/5xx kontrol altında, DB tıkanmıyor.

### 18.1.5 Plugin Tests (Unreal)

Unreal için klasik unit test zor; bunun yerine:

- “Fake backend” (local endpoint) ile test
- “Simülasyon haritası” (örnek level) açılır:
  - 1000 event enqueue
  - flush
  - paket gönderimi doğrulanır

#### Kabul kriteri:

- Oyun akışında FPS hissedilir düşüş yok
- Enqueue sırasında hitch yok
- Network kapalıyken disk spool’a yazıyor
- Network açılınca gönderiyor

### 18.1.6 Frontend Tests

MVP’de aşırı test yazmak yerine:

- 3–5 kritik e2e smoke (Playwright):
  - login
  - project create
  - schema import
  - dashboard aç
  - quarantine görüntüle

#### Kabul kriteri:

- Büyük release öncesi temel akış kırık değil.

### 18.1.7 AI Tests (Hallucination guard)

AI “tavsiye vermez” ve “kanıtsız yorum yapmaz” hedefi için:

- Backend AI prompt çıktısını JSON istiyor
- JSON parse başarısızsa rapor reject + log
- Evidence metrikleri olmadan finding kabul edilmez

### Golden tests:

- Sabit agregat input -> beklenen JSON şeması

### Kabul kriteri:

- AI yanıtı format dışı ise sistem bozulmuyor, rapor “failed” işaretleniyor.

## 19) Dağıtım, Operasyon ve Gözlemlenebilirlik (Minimum)

Sen “grafana şart değil” dedin; buna göre minimal ama yeterli:

### 19.1 Logging

Backend:

- request id (correlation id)
- ingestion summary log: projectId, env, received/accepted/invalid
- quarantine reason histogram (per hour)

Frontend:

- Sentry (opsiyonel ama önerilir)

### 19.2 Monitoring (minimum)

- Health endpoint: /actuator/health
- Metrics endpoint: /actuator/metrics (Prometheus opsiyon)
- DB bağlantı sayısı, slow query log

### 19.3 Deploy hedefi (senin tercihin)

- Frontend: **Vercel**
- Backend: **Railway**
- DB: Railway Postgres / Neon / Supabase (fark etmez; Postgres managed olmalı)

**Kritik not:**

Railway üzerinde ingestion yükü artarsa DB IO tıkanabilir. Partition + indeks ve batch insert şart.

## 20) GELİŞTİRME ADIMLARI (Sıra ile, detaylı uygulama planı)

Aşağıdaki adımlar “Codex’e verilecek” şekilde net ve sırayla yazıldı. Her adımda **çıktı** ve **kabul kriteri** var.

### 20.1 Repo ve klasör yapısı (Windows odaklı)

Öneri: Tek repo (monorepo)

```
telemetryai/  
  backend/           (Spring Boot)  
  frontend/          (React)  
  plugins/  
    unreal/  
    unity/           (opsiyon)  
  infra/  
    docker/  
      docker-compose.yml  
  docs/  
    schema_examples/  
    payload_examples/
```

**Çıktı:**

- Repo açıldı, klasörler hazır.

**Kabul kriteri:**

- `git status` temiz, temel README var.

## 20.2 Local geliştirme ortamı (Docker Compose)

Windows'ta Docker Desktop varsayımı.

`infra/docker/docker-compose.yml`:

- postgres (user/pass/db)
- pgadmin (opsiyon)

**Çıktı:**

- `docker compose up -d` ile Postgres ayağa kalkıyor.

**Kabul kriteri:**

- DB'ye bağlanabiliyorsun.

## 20.3 Backend skeleton (Spring Boot)

Backend başlangıç modülleri:

- `spring-boot-starter-web`
- `spring-boot-starter-security`
- `spring-boot-starter-validation`
- `spring-boot-starter-data-jpa`
- Flyway
- PostgreSQL driver
- Actuator

**İlk hedef:**

- `/actuator/health` çalışsın
- DB connection sağlansın
- Flyway migration çalışsın



**Çıktı:**

- Backend ayağa kalktı.

**Kabul kriteri:**

- Uygulama 0 hata ile başlıyor.

## 20.4 DB migrations (Flyway) — temel tablolar

Önce bunlar:

- orgs, users, org\_members
- projects
- api\_keys
- schemas
- events (partition base)
- quarantine\_events
- ai\_reports
- daily\_aggregates tabloları (boş da olabilir)

**Çıktı:**

- DB şeması oluştu.

**Kabul kriteri:**

- Migration tekrar çalışınca bozulmuyor.

## 20.5 Auth (login) + RBAC (Admin/Viewer)

MVP auth stratejisi:

- Email + password (hash)
- JWT
- Role check (Admin/Viewer)

**Çıktı:**

- Login endpoint + me endpoint çalışıyor.

**Kabul kriteri:**

- Viewer admin endpoint'e giremiyor.

## 20.6 Org/Project/Environment yönetimi

- Org create (opsiyon: sistemde 1 org default)
- Project create/list
- Environment sabit: dev/stage/prod (db'de kayıt)

**Çıktı:**

- UI olmadan Postman ile project oluşturuluyor.

**Kabul kriteri:**

- Project'e sadece org member erişebiliyor.

## 20.7 API Key management (create/rotate/revoke)

- POST /projects/{id}/keys/rotate?env=prod
- DB'de key\_hash sakla
- Rotate: yeni key üret, eskiyi revoke et

**Çıktı:**

- API key üretilip kullanılıyor.

**Kabul kriteri:**

- Revoke edilen key ile ingestion 403 dönüyor.

## 20.8 Schema import/export + current schema

- POST /projects/{id}/schema/import
  - JSON validate
  - schemaVersion++
- GET /projects/{id}/schema/current
- GET /projects/{id}/schema/export

### Çıktı:

- Şema kaydediliyor ve version artıyor.

### Kabul kriteri:

- Bozuk schema import edilemiyor (400 + hata listesi).

## 20.9 Ingestion: /v1/events/batch (en kritik adım)

Bu adımda:

- API key doğrulama
- Body parse
- eventId idempotency
- schema validation
- accepted -> events
- invalid -> quarantine (reasons ile)

### Çıktı:

- Batch endpoint çalışıyor.

### Kabul kriteri:

- Aynı eventId iki kere gelirse duplicate sayılıyor.
- Schema dışı event quarantine'ye gidiyor.

## 20.10 Quarantine API + nedenlerin standardizasyonu

- GET /projects/{id}/quarantine
- reasons enum standard:
  - EVENT\_NOT\_IN\_SCHEMA
  - MISSING\_REQUIRED\_PROPERTY:<key>
  - TYPE\_MISMATCH:<key>
  - PAYLOAD\_TOO\_LARGE
  - PII\_DETECTED

### Çıktı:

- Panelde gösterecek API hazır.

### Kabul kriteri:

- Quarantine kayıtları filtrelenebiliyor.

## 20.11 Aggregation jobs (daily metrics + funnel)

Minimum tablo doldurma:

- daily\_event\_name\_metrics
- daily\_level\_funnel

Job:

- Son 24 saatin event'lerini tarar (partition)
- Özet tabloya yazar (upsert)

### Çıktı:

- Dashboard verisi DB'de hazır.

### Kabul kriteri:

- Aynı gün job tekrar çalışınca double count yok (idempotent upsert).

## 20.12 Dashboard API (Overview + Funnel)

- GET /dashboard/overview
- GET /dashboard/funnel

### Çıktı:

- Frontend'in çizeceği veriler geliyor.

### Kabul kriteri:

- Range parametresi (7d/30d) çalışıyor.

## 20.13 AI Report pipeline (daily + weekly)

1. AI input builder:
  - a. agregat metrikleri topla
  - b. buildVersion kırımını ekle
  - c. quarantine top reasons ekle
2. explanation alanı:
  - a. project bazlı saklanabilir (UI'dan düzenlenebilir)
3. OpenAI çağrısı:
  - a. JSON output istenir
  - b. parse + validate
4. ai\_reports tablosuna yaz

### Çıktı:

- Her gün/hafta rapor üretiliyor.

### Kabul kriteri:

- AI başarısızsa rapor "failed" ama sistem ayakta.

## 20.14 Frontend (React) uygulaması — minimum akış

Sıra:

1. Login
2. Project list/create
3. API key ekranı
4. Schema import/export ekranı
5. Dashboard (overview + funnel)
6. Quarantine ekranı
7. AI reports ekranı

**Çıktı:**

- Panel üzerinden temel kullanım tamam.

**Kabul kriteri:**

- Kullanıcı key'i kopyalayıp plugin'e koyabiliyor.

## 20.15 Unreal Plugin — POC → MVP

Sıra:

1. Plugin skeleton (module, uplugin)
2. Config screen + init
3. TrackEvent: enqueue only
4. Background flush timer
5. HTTP sender + retry/backoff
6. gzip (opsiyon)
7. Offline spool (disk)
8. Shutdown flush
9. Blueprint nodes + C++ API
10. Demo project + örnek level

**Çıktı:**

- Unreal demo oyun event gönderiyor.

**Kabul kriteri:**

- Oyun akışında hitch yok.
- Network kapalıyken spool çalışıyor.
- Network açılınca backlog gönderiliyor.

## 20.16 Release hazırlığı (paketleme + örnekler)

- Schema örnekleri (docs/schema\_examples)
- Payload örnekleri
- “5 dakikada kurulum” yönergesi
- Unreal marketplace paket yapısı

**Çıktı:**

- Satılabilir paket + dokümantasyon.

**Kabul kriteri:**

- Temiz kurulum yapan biri 10 dk içinde event atabiliyor.

## Ekler (Örnekler ve Kurallar)

### Ek A) Event isimlendirme

- lowercase + underscore: level\_end
- custom event’lerde öneri prefix: custom\_

### Ek B) properties kuralları

- sayısal metrikler number
- bool alanlar boolean
- büyük JSON objeleri yasak (limit)

## Ek C) Funnel açıklaması (net)

“Funnel” = oyuncunun adım adım ilerlediği akış.

MVP’de funnel: level\_start -> level\_end

- Level 2 bitirenlerden kaçısı Level 3’e başladı?
- Level 2 sonrası quit oranı kaç?